

Relatório Final

Ian Gabriel Braga Trinta

Caio Possidio Vieira

August 9, 2024

1 Introdução ao Problema

Este documento apresenta o progresso final do projeto de localização para a disciplina de Comunicações Móveis, ministrada pelo professor Daniel Carvalho da Cunha. O objetivo desse relatório é ressaltar o estado final do trabalho, visando destoar um pouco do primeiro relatório, embora ainda resuma os principais pontos de nosso trabalho parcial. Para a estratégia da trilateralização, trabalharemos com as soluções em aprendizado de máquina KNN e Random Forest.

2 Ferramentas Utilizadas

- **Scikit-learn (sklearn)**: Utilizado para a implementação dos algoritmos Random Forest e K-Nearest Neighbors (KNN), além das métricas de avaliação de modelos.
- **Pandas**: Utilizado para a leitura e manipulação dos arquivos CSV.
- **Matplotlib**: Utilizado para a criação de gráficos estatísticos e visualização de dados.
- **NumPy**: Utilizado para operações matemáticas e manipulação de arrays.
- **CTGAN**: Utilizado para trabalharmos com a geração de novos pontos de dados.

3 Tratamento e Preparação dos Dados

Inicialmente, realizamos a etapa de tratamento e preparação dos dados de treinamento e teste, fundamentais para a correta alimentação do modelo preditivo. Esse processo incluiu a integração dos datasets existentes com o arquivo *Bts.csv* por meio da operação de *merge*. Em seguida, verificamos a presença de valores duplicados ou faltantes, confirmando a ausência de ambos. Na sequência, conduzimos uma análise de outliers para identificar possíveis anomalias que poderiam comprometer a performance do modelo. Após a detecção desses outliers, aplicamos os tratamentos necessários, cujos resultados foram visualizados através de boxplots e histogramas. Para finalizar a preparação dos dados, realizamos a normalização dos valores, garantindo a uniformização das escalas e, conseqüentemente, facilitando o aprendizado do modelo. Os dados normalizados foram então salvos em novos arquivos CSV, denominados *train_normalized_dataset* e *test_normalized_dataset*, e prontos para as fases de treinamento e teste do modelo.

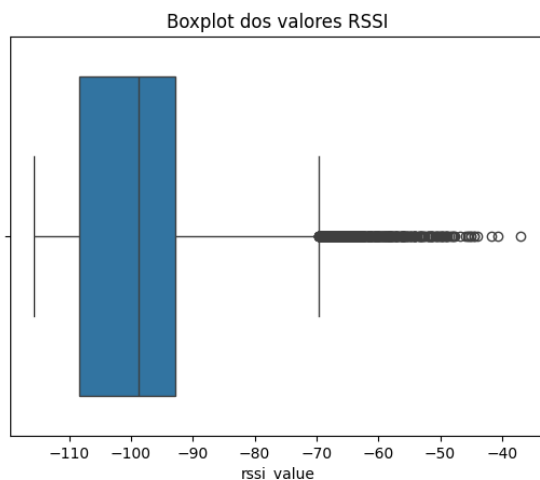


Figure 1: Boxplot - 590 outliers

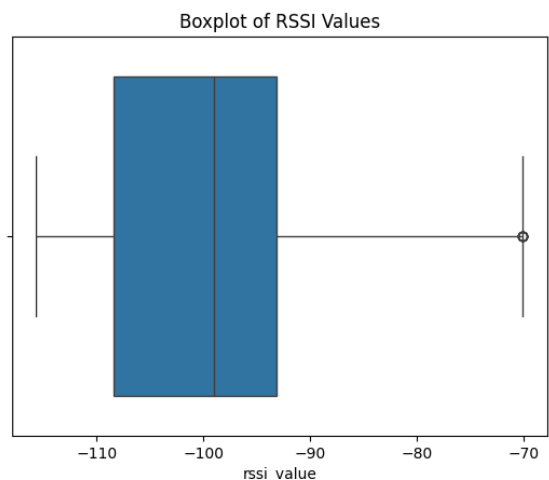


Figure 2: Box Plot após tratamento, apenas 8 outliers

4 Aprendizado de Máquina - Sklearn

4.1 KNN

Utilizamos inicialmente o algoritmo K-Nearest Neighbors (KNN) para realizar cálculos de trilateração. Combinando este método com a técnica de *GridSearchCV* do *scikit-learn*, treinamos o modelo utilizando os melhores hiperparâmetros identificados. Os resultados obtidos, expressos em termos de precisão na localização, são apresentados a seguir:

- **Erro Mínimo de Localização:** 0.00 metro
- **Erro Médio de Localização:** 12.53 metros
- **Erro Máximo de Localização:** 260.82 metros
- **Desvio Padrão do Erro de Localização:** 25.07 metros

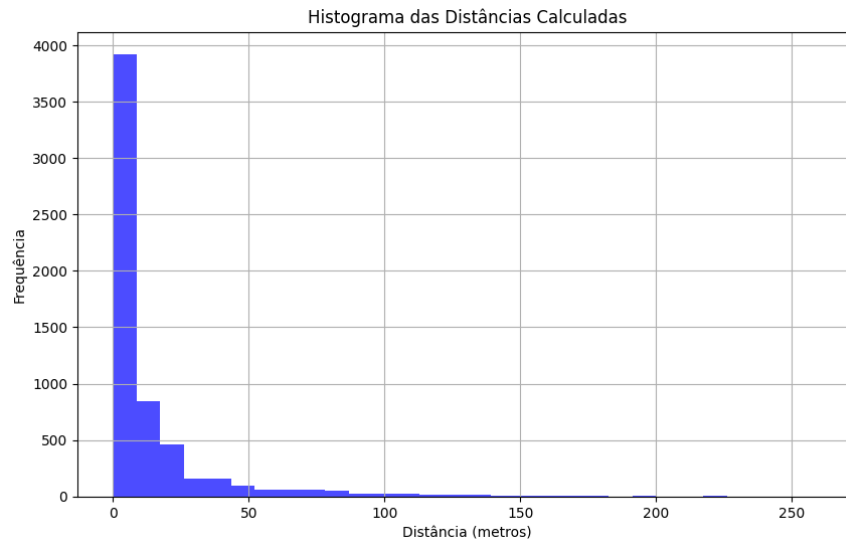


Figure 3: gráfico resultado da aplicação do algoritmo KNN

4.2 Random Forests

Como segundo método, empregamos o algoritmo de Random Forests, utilizando novamente a estratégia de *GridSearchCV* para otimizar os hiperparâmetros. A eficácia do modelo ajustado foi avaliada com base nas seguintes métricas de distância:

- **Erro Mínimo na Distância:** 8.84×10^{-10} metros
- **Erro Médio na Distância:** 13.36 metros
- **Erro Máximo na Distância:** 271.42 metros
- **Desvio Padrão do Erro na Distância:** 26.08 metros

5 Geração de Dados Sintéticos com IA

Para ampliar o conjunto de dados disponível, utilizamos a técnica de IA generativa conhecida como CTGAN (Conditional Tabular GAN) para criar novos dados sintéticos. Inicialmente, preparamos os dados de treino previamente normalizados, aplicando o *LabelEncoder* para transformar as variáveis categóricas em valores numéricos, possibilitando sua utilização no CTGAN. O modelo foi então treinado durante 500 épocas, gerando uma amostra de 30.000 novos registros. Em seguida, revertermos a codificação aplicada pelo *LabelEncoder* para restaurar os valores originais das variáveis categóricas, concluindo assim o processo de geração de dados.

6 Complexidade Temporal

Acerca do tempo de execução do treinamento dos modelos selecionados houve uma grande disparidade de tempo entre o KNN e o Random Forest, sendo 5.53 minutos e 92.22 minutos, seus respectivos tempos de execução no *GridSearchCV*, ainda levantando que foram testadas 600 combinações para o KNN e apenas 300 para o *RandomForest*, o que demonstra ainda mais tal diferença. Outro modelo usado como um complemento foi o CTGAN, que foi citado anteriormente, esse teve um tempo de execução também considerável de 41.16 minutos, devido ao fato de que o treinamos durante 500 épocas.

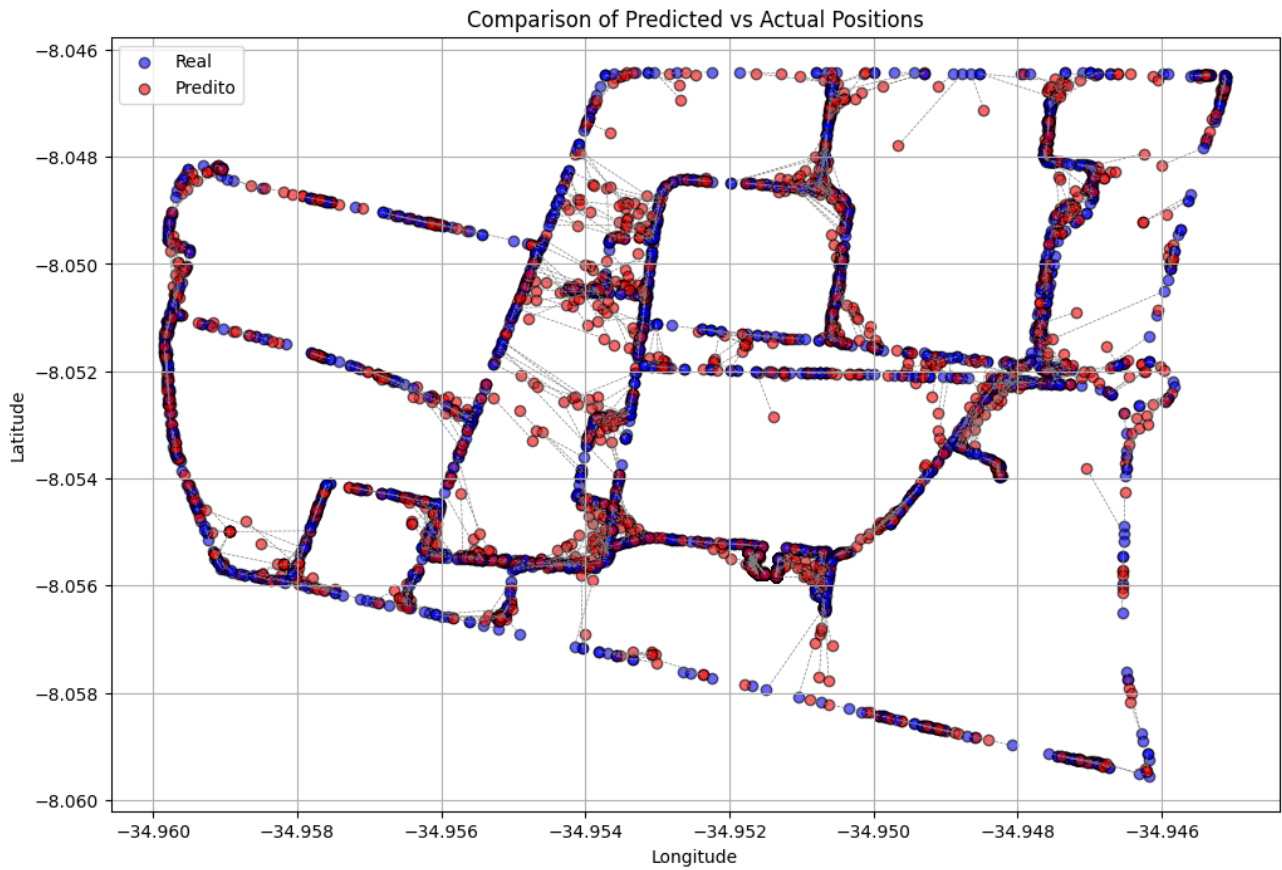


Figure 4: gráfico resultado da aplicação do algoritmo KNN

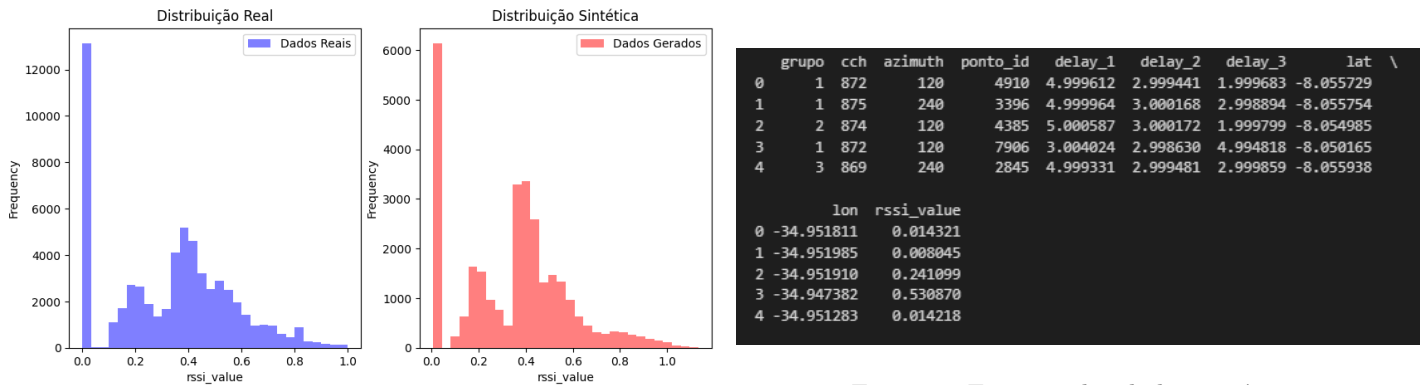


Figure 6: Formato dos dados sintéticos

Figure 5: Histograma comparativo entre os dados

7 Conclusões

7.1 Resumo

Portanto, concluímos que o melhor modelo para a localização de usuários é pelo uso do modelo KNN, visto que este nos retornou o melhor custo benefício na relação resultado/tempo de execução com um erro médio de 12 metros.

7.2 Dificuldades Encontradas

Durante o desenvolvimento do projeto, enfrentamos algumas dificuldades significativas, especialmente no que se refere às limitações computacionais. Destacam-se os seguintes desafios:

- **Escalação da CTGAN:** O treinamento da Conditional Generative Adversarial Network (CTGAN) apresentou tempos extremamente longos, resultando frequentemente em instabilidades e falhas, incluindo o *crash* do kernel.
- **Grid Search para Random Forests:** A utilização da técnica de Grid Search para otimizar os parâmetros do modelo Random Forests enfrentou desafios similares, com problemas de escalabilidade devido à intensidade computacional requerida durante a busca pelos melhores hiperparâmetros.
- **Incorporação de Dados Sintéticos ao KNN:** Ao adicionar dados sintéticos gerados ao dataset utilizado para treinar o modelo K-Nearest Neighbors (KNN), observamos uma piora significativa nas métricas iniciais.